

likewise for a *proper suffix*. The empty string ϵ is both a suffix and a prefix of every string. For any strings x and y and any character a , we have $x \sqsubset y$ if and only if $xa \sqsubset ya$. The $\sqsubset$ and $\sqsupset$ relations are transitive. The following lemma will be useful later.

Lemma 32.1 (Overlapping-suffix lemma)

Suppose that x , y , and z are strings such that $x \sqsubset z$ and $y \sqsubset z$. If $|x| \leq |y|$, then $x \sqsubset y$. If $|x| \geq |y|$, then $y \sqsubset x$. If $|x| = |y|$, then $x = y$.

Proof See Figure 32.2 for a graphical proof. ■

For convenience, denote the k -character prefix $P[1:k]$ of the pattern $P[1:m]$ by $P[:k]$. Thus, we can write $P[:0] = \epsilon$ and $P[:m] = P = P[1:m]$. Similarly, denote the k -character prefix of the text T by $T[:k]$. Using this notation, we can state the string-matching problem as that of finding all shifts s in the range $0 \leq s \leq n - m$ such that $P \sqsubset T[s + m]$.

Our pseudocode allows two equal-length strings to be compared for equality as a primitive operation. If the strings are compared from left to right and the comparison stops when a mismatch is discovered, we assume that the time taken by such a test is a linear function of the number of matching characters discovered. To be precise, the test “ $x == y$ ” is assumed to take $\Theta(t)$ time, where t is the length of the longest string z such that $z \sqsubset x$ and $z \sqsubset y$.

32.1 The naive string-matching algorithm

The NAIVE-STRING-MATCHER procedure finds all valid shifts using a loop that checks the condition $P[1:m] = T[s+1:s+m]$ for each of the $n-m+1$ possible values of s .

```

NAIVE-STRING-MATCHER( $T, P, n, m$ )
1  for  $s = 0$  to  $n - m$ 
2      if  $P[1:m] == T[s + 1:s + m]$ 
3          print “Pattern occurs with shift”  $s$ 

```

Figure 32.3 portrays the naive string-matching procedure as sliding a “template” containing the pattern over the text, noting for which shifts all of the characters on the template equal the corresponding characters in the text. The **for** loop of lines 1–3 considers each possible shift explicitly. The test in line 2 determines whether the current shift is valid. This test implicitly loops to check corresponding character positions until all positions match successfully or a mismatch is found. Line 3 prints out each valid shift s .

Procedure NAIVE-STRING-MATCHER takes $O((n - m + 1)m)$ time, and this bound is tight in the worst case. For example, consider the text string a^n (a string of n a’s) and the pattern a^m . For each of the $n - m + 1$ possible values of the shift s , the implicit loop on line 2 to compare corresponding characters must execute m times to validate the shift. The worst-case running time is thus $\Theta((n - m + 1)m)$, which is $\Theta(n^2)$ if $m = \lfloor n/2 \rfloor$. Because it requires no preprocessing, NAIVE-STRING-MATCHER’s running time equals its matching time.

NAIVE-STRING-MATCHER is far from an optimal procedure for this problem. Indeed, this chapter will show that the Knuth-Morris-Pratt algorithm is much better in the worst case. The naive string-matcher is inefficient because it entirely ignores information gained about the text for one value of s when it considers other values of s . Such information can be quite valuable, however. For example, if $P = \text{aaab}$ and $s = 0$ is valid, then none of the shifts 1, 2, or 3 are valid, since $T[4] = \text{b}$. The following sections examine several ways to make effective use of this sort of information.

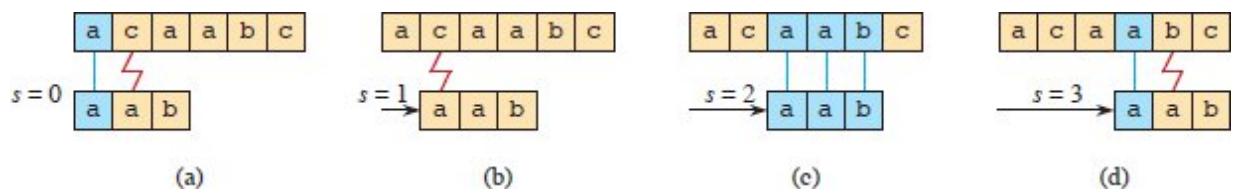


Figure 32.3 The operation of the NAIVE-STRING-MATCHER procedure for the pattern $P = aab$ and the text $T = acaabc$. Imagine the pattern P as a template that slides next to the text. (a)–(d) The four successive alignments tried by the naive string matcher. In each part, vertical lines connect corresponding regions found to match (shown in blue), and a red jagged line connects the first mismatched character found, if any. The algorithm finds one occurrence of the pattern, at shift $s = 2$, shown in part (c).

Exercises

32.1-1

Show the comparisons the naive string matcher makes for the pattern $P = 0001$ in the text $T = 000010001010001$.

32.1-2

Suppose that all characters in the pattern P are different. Show how to accelerate NAIVE-STRING-MATCHER to run in $O(n)$ time on an n -character text T .

32.1-3

Suppose that pattern P and text T are *randomly* chosen strings of length m and n , respectively, from the d -ary alphabet $\Sigma_d = \{0, 1, \dots, d-1\}$, where $d \geq 2$. Show that the *expected* number of character-to-character comparisons made by the implicit loop in line 2 of the naive algorithm is

$$(n - m + 1) \frac{1 - d^{-m}}{1 - d^{-1}} \leq 2(n - m + 1)$$

over all executions of this loop. (Assume that the naive algorithm stops comparing characters for a given shift once it finds a mismatch or matches the entire pattern.) Thus, for randomly chosen strings, the naive algorithm is quite efficient.

32.1-4

Suppose that the pattern P may contain occurrences of a *gap character* $\diamond$ that can match an *arbitrary* string of characters (even one of 0 length). For example, the pattern $ab \diamond ba \diamond c$ occurs in the text $cabccbacbacab$ as

$c \underbrace{ab}_{ab} \underbrace{cc}_{\diamond} \underbrace{ba}_{ba} \underbrace{cba}_{\diamond} \underbrace{c}_{c} \underbrace{ab}_{}$

and as

$c \underbrace{ab}_{ab} \underbrace{ccbac}_{\diamond} \underbrace{ba}_{ba} \underbrace{\quad}_{\diamond} \underbrace{c}_{c} \underbrace{ab}_{}$

The gap character may occur an arbitrary number of times in the pattern but not at all in the text. Give a polynomial-time algorithm to determine whether such a pattern P occurs in a given text T , and analyze the running time of your algorithm.

32.2 The Rabin-Karp algorithm

Rabin and Karp proposed a string-matching algorithm that performs well in practice and that also generalizes to other algorithms for related problems, such as two-dimensional pattern matching. The Rabin-Karp algorithm uses $\Theta(m)$ preprocessing time, and its worst-case running time is $\Theta((n-m+1)m)$. Based on certain assumptions, however, its average-case running time is better.

This algorithm makes use of elementary number-theoretic notions such as the equivalence of two numbers modulo a third number. You might want to refer to Section 31.1 for the relevant definitions.

For expository purposes, let's assume that $\Sigma = \{0, 1, 2, \dots, 9\}$, so that each character is a decimal digit. (In the general case, you can assume that each character is a digit in radix- d notation, so that it has a numerical value in the range 0 to $d - 1$, where $d = |\Sigma|$.) You can then view a string of k consecutive characters as representing a length- k decimal number. For example, the character string 31415 corresponds to the decimal number 31,415. Because we interpret the input